

FoodGenome AI

খাবারের ছবি থেকে পুষ্টি বিশ্লেষণ – সম্পূর্ণ প্রযুক্তিগত ব্যাখ্যা

৯৭.১৬%	Food-101 টেস্ট সেটে top-1 accuracy
৯৯.৬১%	মাপা conformal coverage (গড় ১.৩৯টি প্রার্থী)
৯৮.৫৭%	RAG উত্তর সঠিক (৭৬টি gold কেস)
১০১	খাবারের শ্রেণি, প্রতিটিতে ৩২টি পুষ্টি উপাদান

AI/ML কোর্স প্রজেক্ট

লাইভ ওয়েবসাইট: food-red-omega.vercel.app

রিপোজিটরি: github.com/A-K-M-Asifuzzaman/Food

সূচিপত্র

1. প্রজেক্টটি আসলে কী
2. সমস্যা ও নকশার মূল সিদ্ধান্ত
3. সম্পূর্ণ সিস্টেম আর্কিটেকচার
4. ডেটাসেট: Food-101
5. ডেটা বিভাজন – সবচেয়ে গুরুত্বপূর্ণ ধাপ
6. ধাপ ১: Feature Bank
7. ধাপ ২: Probe Head প্রশিক্ষণ
8. ধাপ ৩: Fine-tuning
9. ধাপ ৪: Ensemble ও McNemar পরীক্ষা
10. ধাপ ৫: Calibration
11. ধাপ ৬: Conformal Prediction
12. ধাপ ৭: Explainability (Grad-CAM)
13. ধাপ ৮: পুষ্টি জ্ঞানভাণ্ডার (USDA)
14. ধাপ ৯: Advanced RAG
15. ধাপ ১০: RAG মূল্যায়ন
16. ব্যাকএন্ড: FastAPI
17. ফ্রন্টএন্ড: Next.js
18. ডিপ্লয়মেন্ট
19. সম্পূর্ণ ফলাফল
20. সীমাবদ্ধতা
21. পরিভাষা

১. প্রজেক্টটি আসলে কী

একজন ব্যবহারকারী তার প্লেটের একটি ছবি তোলেন। সিস্টেমটি পাঁচটি জিনিস ফেরত দেয়:

1. **খাবারটি কী** – ১০১টি শ্রেণির মধ্যে একটি (যেমন `sushi`, `lasagna`)।
2. **কতটা নিশ্চিত** – এমন একটি সংখ্যা যার প্রকৃত অর্থ আছে। "৮০% নিশ্চিত" বললে বাস্তবে প্রায় ৮০% ক্ষেত্রেই সঠিক হয়।
3. **যেসব সম্ভাবনা বাতিল করা যাচ্ছে না** – একটি প্রার্থী-তালিকা, যার সাথে গাণিতিক নিশ্চয়তা যুক্ত।
4. **মডেল কোথায় তাকিয়েছিল** – ছবির উপর একটি তাপচিত্র (heatmap)।
5. **পুষ্টি তথ্য** – ৩২টি উপাদান, প্রতিটির উৎস USDA-র নির্দিষ্ট রেকর্ড পর্যন্ত অনুসরণযোগ্য।

এর সাথে একটি প্রশ্নোত্তর ব্যবস্থা আছে যা কেবল তার জানা তথ্যের ভিত্তিতেই উত্তর দেয়, এবং জানা না থাকলে **উত্তর দিতে অস্বীকার করে**।

কেন এটি সাধারণ প্রজেক্ট থেকে আলাদা

বেশিরভাগ ছাত্র-প্রজেক্ট শুধু accuracy দেখায়। কিন্তু accuracy একা যথেষ্ট নয় – একটি মডেল ৯৭% সঠিক হয়েও প্রতিটি উত্তরে "৯৯.৯% নিশ্চিত" বলতে পারে, তখন ব্যবহারকারী ভালো-খারাপ উত্তরের পার্থক্য বুঝতে পারেন না। এই প্রজেক্টে তাই accuracy-র পাশাপাশি **calibration** (আত্মবিশ্বাস সৎ কি না) এবং **conformal prediction** (গাণিতিক নিশ্চয়তা) যোগ করা হয়েছে।

২. সমস্যা ও নকশার মূল সিদ্ধান্ত

২.১ হার্ডওয়্যারের সীমাবদ্ধতা

পুরো কাজটি একটি **Apple M3 Pro ল্যাপটপ (১৮ GB RAM)**-এ করা হয়েছে। কোনো NVIDIA GPU নেই — PyTorch-এর **MPS** ব্যাকএন্ড ব্যবহার করা হয়েছে, যা Apple-এর নিজস্ব GPU চালায়। এটি একটি A100 GPU-র চেয়ে অনেক ধীর।

এই সীমাবদ্ধতাই সবচেয়ে বড় নকশা-সিদ্ধান্তটি ঠিক করে দিয়েছে: **frozen feature bank**।

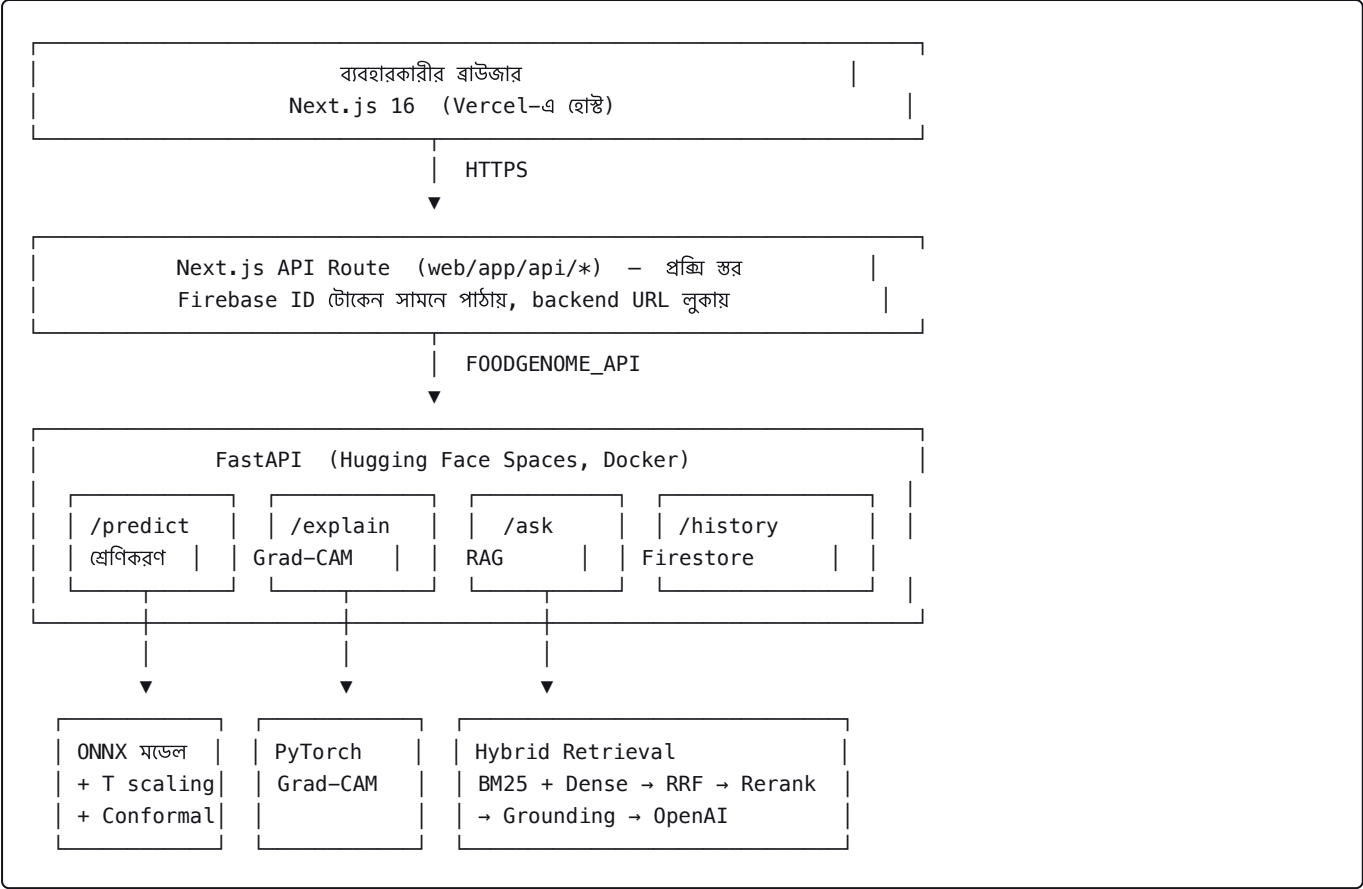
মূল অন্তর্দৃষ্টি

একটি ৩০ কোটি প্যারামিটারের মডেল ল্যাপটপে fine-tune করতে ঘণ্টার পর ঘণ্টা লাগে। প্রতিটি পরীক্ষার জন্য এত সময় দিলে হাতে গোনা কয়েকটি পরীক্ষাই করা যাবে। কিন্তু যদি বড় মডেলটি **একবারই** সব ছবির উপর চালিয়ে ফলাফল (embedding) ডিস্কে জমা রাখি, তাহলে পরবর্তী প্রতিটি পরীক্ষা — head-এর গঠন, calibration, conformal, ensemble — **সেকেন্ডে** শেষ হয়। এই একটি সিদ্ধান্তের কারণেই প্রজেক্টটি বৈজ্ঞানিকভাবে কঠোর হতে পেরেছে।

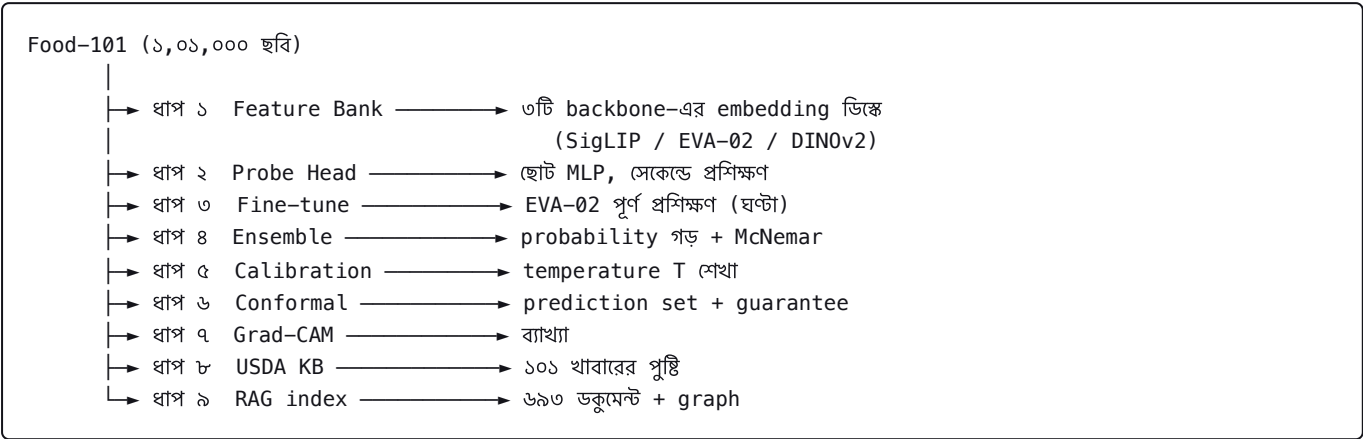
২.২ যেসব নীতি মেনে চলা হয়েছে

নীতি	ব্যাখ্যা
টেস্ট সেট সিল করা	মডেল বাছাই কখনোই টেস্ট সেটে করা হয়নি। train থেকে আলাদা val কেটে নেওয়া হয়েছে।
প্রতিটি সংখ্যা মাপা	ওয়েবসাইটের কোনো সংখ্যা হাতে লেখা নয়; সব স্ক্রিপ্টের লেখা JSON থেকে আসে।
ব্যর্থতাও প্রকাশ	যেসব পরীক্ষা কাজ করেনি সেগুলোও benchmarks পাতায় আছে।
উৎস অনুসরণযোগ্য	প্রতিটি পুষ্টি সংখ্যা USDA-র নির্দিষ্ট <code>fdc_id</code> পর্যন্ত ফিরে যায়।

৩. সম্পূর্ণ সিস্টেম আর্কিটেকচার



প্রশিক্ষণ-পাইপলাইনটি (offline, ল্যাপটপে চলে) সম্পূর্ণ আলাদা:



৪. ডেটাসেট: Food-101

Food-101 একটি বিখ্যাত ডেটাসেট: **১০১টি খাবারের শ্রেণি**, প্রতিটিতে ১,০০০টি ছবি – মোট **১,০১,০০০** ছবি।

বিভাজন	ছবির সংখ্যা	ব্যবহার
train	৭৫,৭৫০	প্রশিক্ষণ (এর ভেতর থেকেই val কাটা হয়)
test	২৫,২৫০	শুধুমাত্র চূড়ান্ত মূল্যায়নে

গুরুত্বপূর্ণ সতর্কতা

Food-101-এর নিজস্ব ডকুমেন্টেশনেই বলা আছে train অংশটি ইচ্ছাকৃতভাবে **পরিষ্কার করা হয়নি** — এতে ভুল লেবেল ও খারাপ ছবি আছে। test অংশটি হাতে পরিষ্কার করা। তাই প্রশিক্ষণে কিছু গোলমাল সহ্য করতে হয়।

৪.১ ডেটা নামানোর কোড

Hugging Face থেকে ডেটা আসে `parquet` ফরম্যাটে। কিন্তু প্রশিক্ষণের সময় বারবার এলোমেলোভাবে ছবি পড়তে হয়, আর `parquet` সেই কাজে ধীর। তাই একবার খরচ করে সব ছবি JPEG হিসেবে ফোল্ডারে লিখে রাখা হয়।

```
src/nutrivision/data/download.py
```

```
# প্রতিটি শ্রেণির জন্য আলাদা ফোল্ডার বানানো হয়
for name in labels:
    (out_split / name).mkdir(parents=True, exist_ok=True)

for idx, row in enumerate(tqdm(ds, desc=f"export {split}")):
    name = labels[row["label"]]
    path = out_split / name / f"{name}_{idx:06d}.jpg"
    if path.exists():
        continue # আগে লেখা থাকলে আবার লিখবে না
    image = row["image"]
    if image.mode != "RGB":
        image = image.convert("RGB") # কিছু ছবি grayscale/RGBA
    image.save(path, "JPEG", quality=95, optimize=True)
```

ফলাফলে ডিস্কে এই কাঠামো তৈরি হয়:

```
data/food101/
├── train/
│   ├── apple_pie/      apple_pie_000000.jpg ... (৭৫০টি)
│   ├── baby_back_ribs/
│   └── ... (১০১টি ফোল্ডার)
└── test/
    └── ... (১০১টি ফোল্ডার, প্রতিটিতে ২৫০টি)
```

৫. ডেটা বিভাজন – সবচেয়ে গুরুত্বপূর্ণ ধাপ

এই প্রজেক্টের সবচেয়ে সাধারণ ভুল যেটি এড়ানো হয়েছে

Food-101-এ কেবল `train` আর `test` আছে – কোনো `validation` নেই। অনেকে তাই টেস্ট সেট দেখে সিদ্ধান্ত নেন ("কোন epoch ভালো?", "কোন মডেল রাখব?")। এতে টেস্টের সংখ্যা **অতিরিক্ত** হয়ে যায়, কারণ টেস্ট সেট আর নিরপেক্ষ থাকে না। এটিকে বলে **data leakage**।

সমাধান: `train` থেকেই ৪% কেটে নিয়ে `val` বানানো হয়েছে। সব সিদ্ধান্ত val-এ, আর test কেবল একদম শেষে একবার।

```
src/nutrivision/data/dataset.py
```

```
def holdout_mask(targets, num_classes, val_fraction, seed=SEED):
    """শ্রেণি-ভিত্তিক (stratified) নির্দিষ্ট validation মাস্ক।

    ছবি-পাইপলাইন আর cached-feature পাইপলাইন – দুটোই একই ফাংশন ব্যবহার
    করে, যাতে দুই জায়গায় একই ছবি val-এ পড়ে। এখানে সামান্য অমিল হলেই
    validation নমুনা প্রশিক্ষণে ঢুকে যাবে এবং পরের সব সংখ্যা ফুলে যাবে।
    """
    rng = np.random.default_rng(seed) # নির্দিষ্ট seed = পুনরাবৃত্তিযোগ্য
    is_val = np.zeros(len(targets), dtype=bool)
    for label in range(num_classes):
        idx = np.flatnonzero(targets == label) # এই শ্রেণির সব সূচক
        rng.shuffle(idx)
        n_val = max(1, int(round(len(idx) * val_fraction)))
        is_val[idx[:n_val]] = True # প্রথম n_val টি val-এ
    return is_val
```

কেন প্রতিটি শ্রেণি আলাদাভাবে (stratified)?

যদি পুরো ডেটা থেকে এলোমেলোভাবে ৪% নিতাম, তাহলে কোনো শ্রেণির হয়তো ৫০টি নমুনা আসত, আবার কোনোটির ২০টি। শ্রেণি-ভিত্তিক ভাগ করলে **প্রতিটি শ্রেণি থেকেই সমান অনুপাতে** নমুনা আসে, ফলে val সেটটি পুরো সমস্যার প্রতিনিধিত্ব করে।

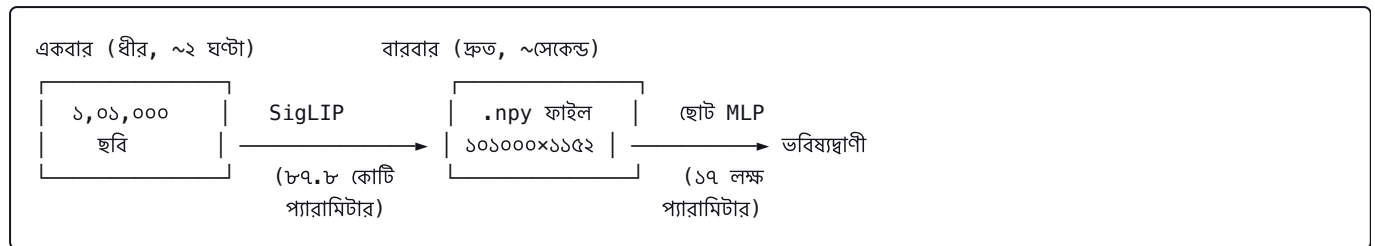
বিভাজন	সংখ্যা	কী কাজে
train (ছাঁটা)	৭২,৭২০	ওজন শেখা
val	৩,০৩০	মডেল বাছাই, temperature, conformal quantile
test	২৫,২৫০	শুধু চূড়ান্ত রিপোর্ট

৬. ধাপ ১: Feature Bank

৬.১ ধারণাটি কী

একটি বড় pretrained মডেল (backbone) একটি ছবিকে একটি সংখ্যার ভেক্টরে রূপান্তর করে — একে বলে **embedding** বা **feature**। এই ভেক্টরে ছবির "অর্থ" ধরা থাকে: দুটি সুশির ছবির ভেক্টর কাছাকাছি হবে, সুশি আর পিৎজার ভেক্টর দূরে থাকবে।

মূল কৌশল: backbone-টিকে **একবারই** সব ছবির উপর চালিয়ে সেই ভেক্টরগুলো ডিস্কে জমিয়ে রাখা। এরপর আর কখনো বড় মডেল চালাতে হয় না।



৬.২ কোন তিনটি backbone এবং কেন

src/nutrivision/config.py

```
FEATURE_BACKBONES = (  
    BackboneSpec(  
        key="siglip_so400m",  
        timm_name="vit_so400m_patch14_siglip_384.webli",  
        image_size=384, dim=1152,  
        notes="Contrastive web-scale pretraining, strongest single "  
            "feature extractor we can run locally.",  
    ),  
    BackboneSpec(  
        key="dinov2_large",  
        timm_name="vit_large_patch14_dinov2.lvd142m",  
        image_size=518, dim=1024,  
        notes="Self-supervised, texture and part sensitive. "  
            "Decorrelated from the contrastive models.",  
    ),  
    BackboneSpec(  
        key="eva02_large",  
        timm_name="eva02_large_patch14_448.mim_m38m_ft_in22k_in1k",  
        image_size=448, dim=1024,  
        notes="Supervised IN-22k finetune, strong fine grained prior.",  
    ),  
)
```

কেন তিনটি আলাদা ধরনের?

তিনটি মডেল **তিনভাবে** প্রশিক্ষিত — SigLIP ছবি-লেখা জোড়া মিলিয়ে (contrastive), DINOv2 কোনো লেবেল ছাড়াই (self-supervised), EVA-02 লেবেল দিয়ে (supervised)। ভিন্ন পদ্ধতিতে শেখা মডেল **ভিন্ন ভুল** করে। একটি যেখানে ভুল করে, অন্যটি সেখানে ঠিক হতে পারে — এই বৈচিত্র্যই ensemble-কে কাজ করায়। একই ধরনের তিনটি মডেল নিলে তারা একই ভুল করত এবং ensemble-এ লাভ হতো না।

৬.৩ feature বের করার কোড

src/nutrivision/models/features.py

```
@torch.no_grad() # gradient লাগবে না — শুধু forward pass
def extract(spec, split, batch_size, workers, image_size, flip, limit=None):
    model, transform, cfg = build_backbone(spec, image_size)
    ds = Food101Folder(split=split, transform=transform, val_fraction=0.0)
    loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
                        num_workers=workers)

    # float16 — অর্ধেক জায়গা, নির্ভুলতায় কার্যত কোনো ক্ষতি নেই
    feats = np.zeros((len(ds), spec.dim), dtype=np.float16)
    labels = np.zeros(len(ds), dtype=np.int64)
    cursor = 0

    autocast = torch.autocast(device_type=DEVICE.type, dtype=torch.float16,
                              enabled=DEVICE.type != "cpu")
    for images, target in tqdm(loader):
        images = images.to(DEVICE, non_blocking=True)
        with autocast:
            out = model(images) # (B, dim) — pooled embedding
            if flip: # বিকল্প: আয়না-ছবির সাথে গড়
                out = (out + model(torch.flip(images, dims=[3]))) / 2
            n = out.shape[0]
            feats[cursor:cursor + n] = out.float().cpu().numpy().astype(np.float16)
            labels[cursor:cursor + n] = target.numpy()
            cursor += n

    np.save(out_dir / f"{split}_x.npy", feats) # ভেক্টর
    np.save(out_dir / f"{split}_y.npy", labels) # লেবেল
```

কোডের গুরুত্বপূর্ণ অংশগুলো

- `@torch.no_grad()` — এখানে শেখা হচ্ছে না, তাই gradient হিসাব বন্ধ। এতে মেমরি ও সময় দুটোই বাঁচে।
- `num_classes=0` (`build_backbone` -এ) — মডেলের শেষ শ্রেণিকরণ স্তরটি বাদ দেওয়া হয়, যাতে সরাসরি ভেক্টর পাওয়া যায়।
- `shuffle=False` — ক্রম অপরিবর্তিত রাখা **অপরিহার্য**, কারণ পরে এই ক্রম ধরেই লেবেলের সাথে মেলানো হবে।
- `float16` — ১,০১,০০০ × ১১৫২ সংখ্যা float32-এ ৪৬৫ MB, float16-এ ২৩২ MB।

৬.৪ ফলাফল

ডিস্কে তৈরি হয় (SigLIP-এর উদাহরণ):

```
artifacts/features/siglip_so400m/
├── train_x.npy    ১৭৪ MB    (৭৫,৭৫০ × ১১৫২)
├── train_y.npy    ০.৬ MB
├── test_x.npy     ৫৮ MB     (২৫,২৫০ × ১১৫২)
└── test_y.npy     ০.২ MB
```

৭. ধাপ ২: Probe Head প্রশিক্ষণ

এখন জমানো ভেক্টরের উপর একটি ছোট নিউরাল নেটওয়ার্ক প্রশিক্ষণ দেওয়া হয় যা ভেক্টর থেকে ১০১টি শ্রেণির একটিতে ম্যাপ করে। একে বলে **probe head**।

৭.১ সরল MLP

src/nutrivision/models/heads.py

```
class MLPProbe(nn.Module):
    def __init__(self, dim, num_classes, hidden=1536, dropout=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.LayerNorm(dim),          # ইনপুট স্বাভাবিক করা
            nn.Linear(dim, hidden),     # ১১৫২ → ১৫৩৬
            nn.GELU(),                  # অ-রৈখিকতা
            nn.Dropout(dropout),        # overfitting রোধ
            nn.LayerNorm(hidden),
            nn.Linear(hidden, num_classes), # ১৫৩৬ → ১০১
        )

    def forward(self, x):
        return self.net(x)
```

৭.২ Gated Fusion — দুটি backbone একসাথে

দুটি backbone-এর ভেক্টর কেবল জোড়া দিলে (concatenate) একটিই ওজন-ম্যাট্রিক্সকে দুটি ভিন্ন স্কেলের স্থান সামলাতে হয়। বদলে, প্রতিটিকে একই আকারে প্রক্ষেপণ করে একটি **gate** শেখানো হয়, যা প্রতিটি ছবির জন্য আলাদাভাবে ঠিক করে কোন backbone-কে কতটা বিশ্বাস করা হবে।

src/nutrivision/models/heads.py

```

class GatedFusionProbe(nn.Module):
    def __init__(self, dims, num_classes, proj=1024, hidden=1536, dropout=0.3):
        super().__init__()
        # প্রতিটি backbone-কে একই প্রক্ষেপ আনা
        self.projections = nn.ModuleList(
            nn.Sequential(nn.LayerNorm(d), nn.Linear(d, proj), nn.GELU())
            for d in dims
        )
        # gate: কোনটিকে কত ওজন - যোগফল ১
        self.gate = nn.Sequential(
            nn.Linear(proj * len(dims), len(dims)),
            nn.Softmax(dim=-1),
        )
        self.head = nn.Sequential(
            nn.LayerNorm(proj), nn.Linear(proj, hidden), nn.GELU(),
            nn.Dropout(dropout), nn.LayerNorm(hidden),
            nn.Linear(hidden, num_classes),
        )

    def forward(self, xs, return_gate=False):
        projected = [p(x) for p, x in zip(self.projections, xs)]
        stacked = torch.stack(projected, dim=1) # (B, M, proj)
        weights = self.gate(torch.cat(projected, -1)).unsqueeze(-1)
        fused = (stacked * weights).sum(dim=1) # ওজনসহ যোগ
        logits = self.head(fused)
        return (logits, weights.squeeze(-1)) if return_gate else logits

```

৭.৩ Embedding-স্থানে Mixup

Mixup একটি নিয়মিতকরণ (regularization) কৌশল: দুটি নমুনা ও তাদের লেবেল মিশিয়ে নতুন প্রশিক্ষণ-উদাহরণ বানানো হয়। ছবি জমে যাওয়ার পর পিক্সেলে mixup করা যায় না, কিন্তু **ভেক্টরে** করা যায় এবং তাতে খরচও নেই।

```

def mixup_features(x, y, num_classes, alpha, smoothing):
    off = smoothing / num_classes
    on = 1.0 - smoothing + off
    targets = torch.full((y.size(0), num_classes), off, device=y.device)
    targets.scatter_(1, y.unsqueeze(1), on) # label smoothing

    if alpha <= 0:
        return x, targets

    lam = float(torch.distributions.Beta(alpha, alpha).sample())
    perm = torch.randperm(x.size(0), device=x.device)
    # ভেক্টর ও লেবেল - দুটোই একই অনুপাতে মেশানো
    return lam * x + (1 - lam) * x[perm], lam * targets + (1 - lam) * targets[perm]

```

৭.৪ ফলাফল

Head	প্যারামিটার	val top-1	test top-1	সময়
SigLIP-SO400M (MLP)	১৯, ৩১, ৬২১	৯৫.৬৪	৯৬.৮৩	৩.৭ মি
EVA-02-L (MLP)	১৭, ৩৪, ৭৫৭	৯৩.১৪	৯৫.৫৩	৬.৯ মি
DINOv2-L (MLP)	১৭, ৩৪, ৭৫৭	৯২.৬৪	৯৪.৮৭	০.৮ মি
Gated Fusion (২টি)	৩৯, ৭৩, ৪৭৯	৯৫.৭৮	৯৬.৯৭	২১.৯ মি

লক্ষণীয়

মাত্র **৩.৭ মিনিটে** ৯৬.৮৩% পাওয়া গেছে – কারণ ভারী কাজটি (feature extraction) আগেই একবার হয়ে গেছে।
এটিই feature bank কৌশলের মূল লাভ।

৮. ধাপ ৩: Fine-tuning

Probe head-এ backbone অপরিবর্তিত থাকে। **Fine-tuning**-এ পুরো backbone-এর ওজনও Food-101-এর জন্য নতুন করে শেখানো হয়। এতে সম্ভাব্য নির্ভুলতা বেশি, কিন্তু খরচ অনেক বেশি।

৮.১ Progressive Resolution

কৌশলটি

উচ্চ রেজোলিউশনে (৪৪৮ পিক্সেল) প্রশিক্ষণ ২২৪ পিক্সেলের চেয়ে প্রায় ৪ গুণ ধীর, কারণ Vision Transformer-এ টোকেন সংখ্যা রেজোলিউশনের বর্গের সমানুপাতিক। সমাধান: **বেশিরভাগ epoch ২২৪-এ** চালানো (যেখানে মডেল শেখে কোন শ্রেণি কী), তারপর **শেষ অল্প কয়েকটি epoch ৪৪৮-এ** (যা প্রশিক্ষণ ও পরীক্ষার রেজোলিউশনের ব্যবধান মেটায়)। ল্যাপটপে এতে পূর্ণ উচ্চ-রেজোলিউশন প্রশিক্ষণের প্রায় সব লাভ এক-তৃতীয়াংশ সময়ে পাওয়া যায়।

৮.২ আধুনিক প্রশিক্ষণ-রেসিপি

src/nutrivision/training/finetune.py

```
train_tf = create_transform(
    **model_cfg, is_training=True,
    auto_augment="rand-m9-mstd0.5-inc1", # RandAugment
    re_prob=0.25, re_mode="pixel", # Random Erasing
    scale=(0.5, 1.0), # এলোমেলো ক্রপ
    hflip=0.5, color_jitter=0.3,
)

mixup = Mixup(
    mixup_alpha=0.8, cutmix_alpha=1.0, # Mixup ও CutMix
    prob=0.5, switch_prob=0.5,
    label_smoothing=0.1, num_classes=101,
)

criterion = SoftTargetCrossEntropy()
```

Layer-wise Learning Rate Decay

Pretrained মডেলের প্রথম স্তরগুলো সাধারণ বৈশিষ্ট্য শেখে (কোণা, রঙ) – সেগুলো নষ্ট করা উচিত নয়। শেষ স্তরগুলো কাজ-নির্দিষ্ট। তাই **প্রথম স্তরে ছোট learning rate, শেষ স্তরে বড়**:

```
return param_groups_layer_decay(
    model, weight_decay=weight_decay,
    layer_decay=layer_decay, # 0.75 – প্রতি স্তরে ২৫% কমে
    no_weight_decay_list=model.no_weight_decay(),
)
```

Cosine schedule + Warmup

```
def lr_scale(step):
    if step < warmup:
        return (step + 1) / warmup          # ধীরে শুরু
    p = (step - warmup) / max(1, total - warmup)
    return 0.5 * (1 + math.cos(math.pi * p)) * (1 - 0.01) + 0.01
```

৮.৩ বাধা-সহনশীলতা

বাস্তব প্রকৌশল সমস্যা

ল্যাপটপে ২০ ঘণ্টার প্রশিক্ষণ **নিশ্চিতভাবেই** কখনো বাধাগ্রস্ত হবে — ঢাকনা বন্ধ, ব্যাটারি শেষ, বা অন্য কাজ। ২০ ঘণ্টা হারানো গ্রহণযোগ্য নয়।

```
_INTERRUPTED = False

def _on_signal(signum, frame):
    global _INTERRUPTED
    _INTERRUPTED = True
    print("interrupt received, finishing this step then checkpointing")

signal.signal(signal.SIGINT, _on_signal)    # Ctrl-C
signal.signal(signal.SIGTERM, _on_signal)   # kill

# ... প্রতিটি epoch শেষে সম্পূর্ণ অবস্থা সংরক্ষণ ...
state = {
    "stage": stage_name, "epoch": epoch,
    "model": model.state_dict(),
    "ema": ema.module.state_dict(),
    "optimizer": opt.state_dict(),        # optimizer-ও লাগে!
    "scheduler": sched.state_dict(),
    "best_acc": max(best_acc, report_acc),
    "history": history,
}
torch.save(state, ckpt_path)
```

`optimizer.state_dict()` সংরক্ষণ করা জরুরি — AdamW প্রতিটি প্যারামিটারের জন্য চলমান গড় রাখে। শুধু ওজন সংরক্ষণ করে পুনরায় শুরু করলে সেই গড় হারিয়ে যায় এবং প্রশিক্ষণ ধাক্কা খায়।

৮.৪ ফলাফল

ধাপ	val top-1	test top-1	সময়
Stage-1 (২২৪ px, ৬ epoch)	৯১.০৬	৯৩.৪৩	২৮.৭ মি/epoch
সম্পূর্ণ fine-tune	—	৯৫.৯০	কয়েক ঘণ্টা

প্রত্যাশার বিপরীত ফলাফল

ঘণ্টার পর ঘণ্টা fine-tuning করে পাওয়া **৯৫.৯০%**, অথচ মাত্র ৩.৭ মিনিটের probe head দিয়ে **৯৬.৮৩%**! অর্থাৎ এই ক্ষেত্রে frozen feature + ছোট head, পূর্ণ fine-tuning-এর চেয়ে ভালো করেছে। কারণ সম্ভবত ছোট ডেটাসেটে বিশাল মডেল fine-tune করলে overfitting হয়, আর SigLIP-এর web-scale pretraining এমনিতেই খাবারের জন্য যথেষ্ট শক্তিশালী।

৯. ধাপ ৪: Ensemble ও McNemar পরীক্ষা

Ensemble মানে একাধিক মডেলের ভবিষ্যদ্বাণী একত্র করা। সহজতম উপায়: প্রতিটি মডেলের probability-র গড় নেওয়া।

```
# সহজ গড় – কোনো শেখা প্যারামিটার নেই  
p_final = (p_siglip + p_eva02) / 2
```

৯.১ ফলাফল

পদ্ধতি	test top-1	test top-5
SigLIP একা	৯৬.৮৩	৯৯.৫৩
Gated Fusion (শেখা, ৩৯.৭ লক্ষ প্যারামিটার)	৯৬.৯৭	৯৯.৪৮
Probability গড় (SigLIP + EVA-02)	৯৭.১৬	৯৯.৬৪
Oracle (যেকোনো একটি ঠিক হলেই)	৯৮.৩০	—

দ্বিতীয় প্রত্যাশা-বিরোধী ফলাফল

৩৯.৭ লক্ষ প্যারামিটারের **শেখা** fusion head (২১.৯ মিনিট প্রশিক্ষণ) হেরে গেছে মিলিসেকেন্ডে হিসাব করা একটি **সাধারণ গড়ের** কাছে। জটিলতা সবসময় ভালো ফল দেয় না – এটি রিপোর্ট করা গুরুত্বপূর্ণ।

৯.২ পার্থক্যটি কি পরিসংখ্যানগতভাবে অর্থবহ?

৯৬.৮৩% আর ৯৭.১৬% – পার্থক্য মাত্র ০.৩৩%। এটি কি সত্যিকারের উন্নতি, নাকি কাকতালীয়? এর উত্তর দেয় **McNemar পরীক্ষা**।

ধারণাটি সরল: শুধু সেইসব ছবি গোনো হয় যেখানে দুই মডেল **ভিন্নমত** পোষণ করে।

	সংখ্যা
শুধু ensemble ঠিক (মডেল A ঠিক, B ভুল)	১৯৮
শুধু SigLIP ঠিক (B ঠিক, A ভুল)	১১৫
p-value	০.০০০০০৩

$p < 0.05$ মানে এই পার্থক্য কাকতালীয় হওয়ার সম্ভাবনা খুবই কম। তাই ensemble সতিই ভালো — significant

১০. ধাপ ৫: Calibration

১০.১ সমস্যা

আধুনিক নিউরাল নেটওয়ার্ক **পদ্ধতিগতভাবে অতি-আত্মবিশ্বাসী**। মডেল "৯৯% নিশ্চিত" বললেও বাস্তবে হয়তো ৯২% ক্ষেত্রে ঠিক। পুষ্টি-অ্যাপে এটি বিপজ্জনক, কারণ ব্যবহারকারী নির্ভরযোগ্য আর অনুমানভিত্তিক উত্তরের পার্থক্য বুঝতে পারেন না।

১০.২ সমাধান: Temperature Scaling

একটি মাত্র সংখ্যা T শেখা হয়। softmax-এর আগে logits-কে T দিয়ে ভাগ করা হয়:

```
p = softmax(logits / T)
```

- $T > 1$ → বিতরণ নরম হয় → আত্মবিশ্বাস কমে
- $T < 1$ → বিতরণ ধারালো হয় → আত্মবিশ্বাস বাড়ে

গুরুত্বপূর্ণ ধর্ম

ভাগ করা সব ক্ষেত্রে সমানভাবে হয়, তাই **সবচেয়ে বড় মান কোনটি তা বদলায় না**। অর্থাৎ accuracy গাণিতিকভাবে **অপরিবর্তিত** থাকে — শুধু আত্মবিশ্বাসের সংখ্যাটি ঠিক হয়। ফলাফলে দেখা যায় দুটোই ৯৭.১৫৬%।

```
src/nutrivision/reliability/calibration.py
```

```
def fit_temperature(logits, y, max_iter=200):
    """validation-এ NLL সর্বনিম্ন করে এমন একটি scalar T খোঁজা।"""
    log_t = torch.zeros(1, requires_grad=True) # log-স্থানে, যাতে  $T > 0$ 
    opt = torch.optim.LBFGS([log_t], lr=0.1, max_iter=max_iter)

    def closure():
        opt.zero_grad()
        loss = F.cross_entropy(logits / log_t.exp(), y)
        loss.backward()
        return loss
    opt.step(closure)
    return float(log_t.exp())
```

১০.৩ ফলাফল

মেট্রিক	আগে	পরে	মানে
top-1 accuracy	৯৭.১৫৬	৯৭.১৫৬	অপরিবর্তিত (প্রত্যাশিত)
ECE	০.০৫১৩৫	০.০০৬২৪	৮ গুণ ভালো
Brier score	০.০৫১৩৬	০.০৪৬০৫	উন্নত
NLL	০.১৫৫৭৯	০.১১৮৭১	উন্নত
শেখা T		০.৭৬২১	মডেল আসলে অতি-সতর্ক ছিল

ECE (Expected Calibration Error) = ঘোষিত আত্মবিশ্বাস আর প্রকৃত নির্ভুলতার গড় পার্থক্য। ০ হলে নিখুঁত। ০.০৫১ → ০.০০৬ মানে এখন "৮০% নিশ্চিত" বললে বাস্তবেও প্রায় ৮০% ঠিক।

১১. ধাপ ৬: Conformal Prediction

১১.১ ধারণাটি কেন শক্তিশালী

দুই ধরনের দাবি

- "আমি ৯৪% নিশ্চিত" – এটি মডেলের **নিজের সম্পর্কে নিজের দাবি**। কেউ যাচাই করেনি।
- "সঠিক উত্তরটি এই তালিকার ভেতরে থাকবে ৯৫% ক্ষেত্রে" – এটি **মাপা ও প্রমাণিত** একটি নিশ্চয়তা।

Conformal prediction একটি একক লেবেলের বদলে একটি **সেট** ফেরত দেয়, এবং গ্যারান্টি দেয় যে সত্য উত্তর সেই সেটে থাকবে অন্তত $(1-\alpha)$ অনুপাতে। এই গ্যারান্টি মডেলের স্থাপত্য বা স্কোর-বিতরণ নিয়ে **কোনো অনুমান করে না**। একমাত্র শর্ত – calibration ও test ডেটা exchangeable হতে হবে।

১১.২ কীভাবে কাজ করে

১. val সেটের প্রতিটি নমুনার "অস্বাভাবিকতা স্কোর" বের করা
LAC: $s = 1 - p(\text{সত্য শ্রেণি})$
২. সেই স্কোরগুলোর $(1-\alpha)$ quantile নেওয়া $\rightarrow \hat{q}$
৩. test-এ: যেসব শ্রেণির স্কোর $\leq \hat{q}$, সবগুলো সেটে রাখা

```
src/nutrivision/reliability/conformal.py
```

```
def conformal_quantile(scores, alpha):  
    n = len(scores)  
    # সসীম নমুনার সংশোধন: ceil((n+1)(1-alpha)) / n  
    level = math.ceil((n + 1) * (1 - alpha)) / n  
    return float(np.quantile(scores, min(level, 1.0), method="higher"))
```

১১.৩ দুটি পদ্ধতি

পদ্ধতি	স্কোর	সুবিধা	অসুবিধা
LAC	$1 - p(\text{সত্য})$	সবচেয়ে ছোট সেট	coverage গড়ে ঠিক, প্রতিটি শ্রেণিতে নয়
APS	ক্রমসম্প্রতি probability	শ্রেণি-ভিত্তিক coverage ভালো	সেট বড় হয়

১১.৪ মাপা ফলাফল

লক্ষ্য coverage	পদ্ধতি	প্রাপ্ত coverage	গড় সেট আকার
৯০%	LAC	৯৩.৬১%	০.৯৫
	APS	৯০.২৮%	১.২৮
৯৫%	LAC	৯৭.০৮%	১.০০
	APS	৯৫.৬৪%	১.৯৩
৯৯%	LAC	৯৯.৬১%	১.৩৯
	APS	৯৯.৩৮%	৫.০২

অর্থাৎ ৯৯% নিশ্চয়তা চাইলে গড়ে মাত্র **১.৩৯টি** প্রার্থী দেখালেই যথেষ্ট – ব্যবহারিকভাবে খুবই ব্যবহারযোগ্য।

১২. ধাপ ৭: Explainability (Grad-CAM)

মডেল ছবির **কোন অংশ** দেখে সিদ্ধান্ত নিল? Grad-CAM এই প্রশ্নের উত্তর দেয়: শেষ স্তরের activation-এর সাপেক্ষে শ্রেণি-স্কোরের gradient নিয়ে একটি তাপচিত্র বানানো হয়।

এই প্রজেক্টে Vision Transformer ব্যবহার হয়েছে, তাই gradient শেষ transformer block থেকে pooling head হয়ে patch-গ্ৰিডে ফেরত নেওয়া হয়।

সততার একটি বিষয়

প্যানেলটি শুধু তাপচিত্র দেখায় না – এটি হিসাব করে দেখায় মনোযোগ কতটা **ঘনীভূত**। যদি তাপচিত্র প্রায় পুরো ছবি জুড়ে ছড়ানো থাকে, তবে সেটিকে "diffuse" লেবেল দিয়ে স্থাপত্যগত কারণ ব্যাখ্যা করা হয়। প্রেক্ষাপট ছাড়া তাপচিত্র দেখানো পাঠককে এমন নির্ভুলতা কল্পনা করায় যা পদ্ধতিটির নেই।

১৩. ধাপ ৮: পুষ্টি জ্ঞানভাণ্ডার (USDA)

ছবি চিনে ফেলার পর প্রশ্ন – এই খাবারে কত ক্যালরি, কত প্রোটিন? এখানে **কোনো সংখ্যা বানানো হয়নি**। সব এসেছে **USDA FoodData Central (SR Legacy)** থেকে, যা মার্কিন কৃষি বিভাগের সরকারি ডেটাবেস – ৭,৭৯৩টি খাবারের সম্পূর্ণ পুষ্টি-প্রোফাইল।

১৩.১ ৩২টি পুষ্টি উপাদান

```
NUTRIENTS = {  
    "energy_kcal": (1008, "Energy", "kcal"),  
    "protein_g": (1003, "Protein", "g"),  
    "fat_g": (1004, "Total fat", "g"),  
    "carbs_g": (1005, "Carbohydrate", "g"),  
    "fiber_g": (1079, "Dietary fiber", "g"),  
    "sodium_mg": (1093, "Sodium", "mg"),  
    "calcium_mg": (1087, "Calcium", "mg"),  
    "iron_mg": (1089, "Iron", "mg"),  
    "vitamin_c_mg": (1162, "Vitamin C", "mg"),  
    # ... মোট ৩২টি: ম্যাক্রো, খনিজ, ভিটামিন  
}
```

১৩.২ মূল সমস্যা: ম্যাপিং

সমস্যাটি

USDA-তে `apple_pie`-এর রেকর্ড আছে। কিন্তু **baklava**, **bibimbap**, **takoyaki**, **beef carpaccio** — এগুলোর কোনো রেকর্ড **নেই**। ১০১টির মধ্যে মাত্র ৪১টির সরাসরি মিল পাওয়া যায়।

দুই স্তরের সমাধান:

পদ্ধতি	সংখ্যা	ব্যাখ্যা
<code>direct</code>	৪১	USDA-তে সরাসরি রেকর্ড আছে
<code>composite</code>	৬০	উপকরণ থেকে ওজন-ভিত্তিক পুনর্গঠন

যেমন baklava-র জন্য: ফিলো পেস্তা + আখরোট + মাখন + মধু + চিনি — প্রতিটির গ্রাম-ওজন সহ USDA রেকর্ড নিয়ে যোগ করা হয়। এটি অনুমান করার চেয়ে **সং**, এবং অতিরিক্ত লাভ — উপকরণের তালিকা পরে অ্যালার্জেন ও বিকল্প-সুপারিশে কাজে লাগে।

১৩.৩ USDA খোঁজার সমস্যা

SR Legacy-তে সাধারণ খাবারের পাশাপাশি বেবিফুড, শুকনো মিষ্টি ও রেস্টোরাঁ-ব্র্যান্ডের এন্ট্রিও আছে। "carrot cake" খুঁজলে প্রথমে আসে *"Babyfood, carrots and beef, strained"* আর *"Cake, pudding-type, carrot, dry mix"* — যা কেউই কেক হিসেবে খাবে না।

```
src/nutrivision/nutrition/resolve.py
```

```

PENALTY_PATTERNS = (
    (re.compile(r"\bbabyfood\b", re.I), 40.0),          # বেবিফুড বাদ
    (re.compile(r"\bdry mix\b|\bunprepared\b", re.I), 12.0),
    (re.compile(r"\binfant\b|\bformula\b|\bstrained\b", re.I), 30.0),
    (re.compile(r"\bpie fillings\b|\bfrosting\b", re.I), 8.0),
)

BRAND_TOKEN = re.compile(r"\b[A-Z]{3,}(:?'?[A-Z]+)?\b") # ব্র্যান্ড নাম
GENERIC_HINT = re.compile(r"\b(home-prepared|prepared from recipe|"
                           r"commercially prepared)\b", re.I)

```

স্কোরিং: শব্দ মিললে +, উপরের প্যাটার্ন মিললে -, বড় হাতের ব্র্যান্ড-টোকেন থাকলে -, "home-prepared" জাতীয় ইঙ্গিত থাকলে +।

১৩.৪ ফলাফল

```

data/nutrition/kb.json
{
  "schema": 1,
  "source": "USDA FoodData Central, SR Legacy 2018-04",
  "num_classes": 101,
  "entries": [ {
    "class": "sushi", "title": "Sushi", "cuisine": "Japanese",
    "serving_g": 180, "method": "composite",
    "fdc_id": ..., "description": "...",
    "nutrients_per_100g": { ... ৩২টি ... },
    "nutrients_per_serving": { ... ৩২টি ... },
    "review_flags": [ ... ]
  }, ... ]
}

```

১৪. ধাপ ৯: Advanced RAG

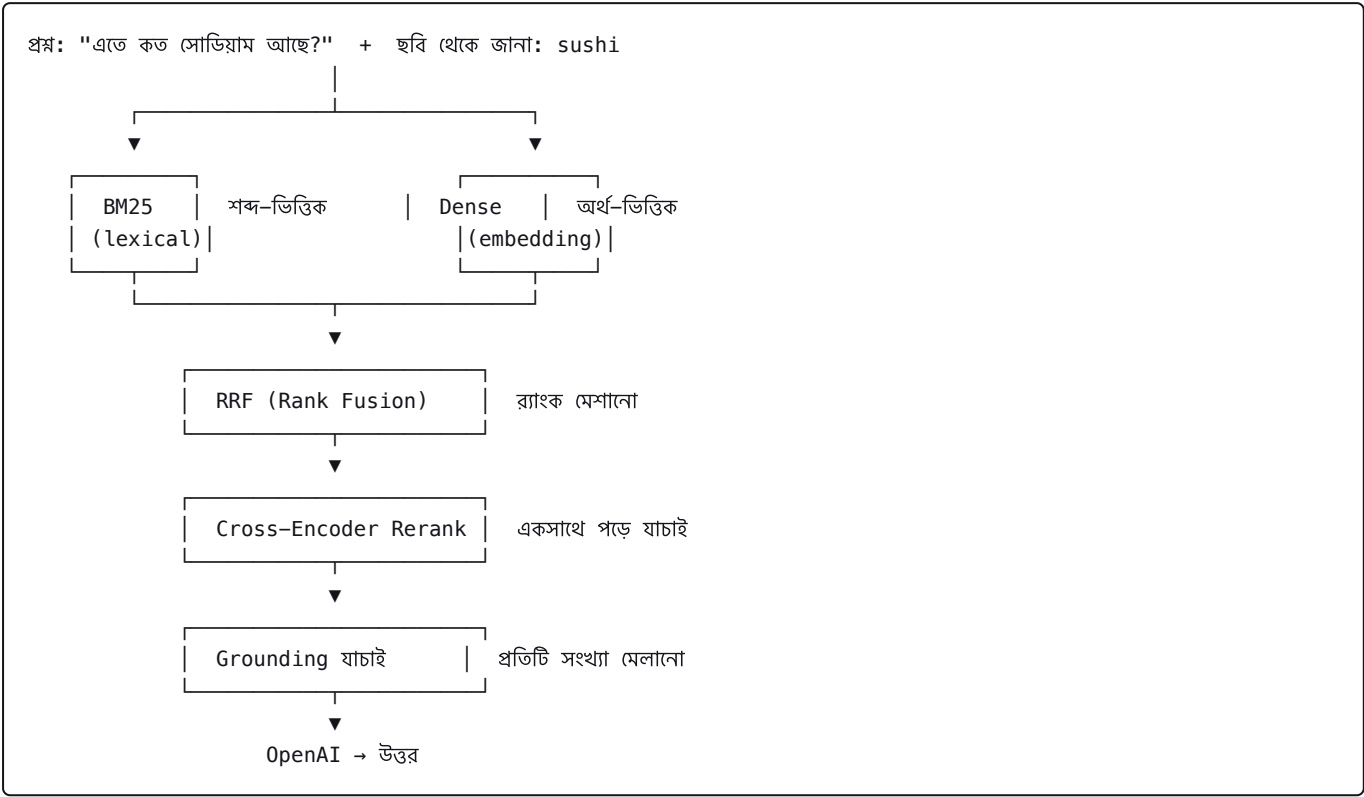
RAG (Retrieval-Augmented Generation) = প্রথমে প্রাসঙ্গিক তথ্য খুঁজে বের করা, তারপর সেই তথ্য দিয়ে ভাষা-মডেলকে উত্তর লেখানো। এতে মডেল **কল্পনা** করতে পারে না, কারণ তাকে শুধু প্রাপ্ত নথি থেকে উত্তর দিতে বলা হয়।

১৪.১ কর্পাস তৈরি

জ্ঞানভাণ্ডার থেকে **৬৯৩টি** ডকুমেন্ট তৈরি হয়, নয় ধরনের:

ধরন	উদাহরণ
identity	খাবারটি কী, কোন রন্ধনশৈলী
macros	ক্যালরি, প্রোটিন, চর্বি, শর্করা
minerals / vitamins	খনিজ ও ভিটামিন
ingredients	composite খাবারের উপকরণ
portion	পরিবেশন আকার
ranking	"সবচেয়ে বেশি প্রোটিন" জাতীয়
graph_ingredient / graph_similarity	গ্রাফ থেকে উৎপন্ন তথ্য

১৪.২ Hybrid Retrieval – তিন স্তর



কেন RRF, সরাসরি স্কোর যোগ নয়?

BM25 অসীম-সীমার স্কোর দেয়; cosine similarity -১ থেকে $+১$ -এর মধ্যে। এদের ওজনসহ যোগ করতে গেলে একটি normalization দরকার, যা নিজেই একটি tune করার প্যারামিটার এবং কর্পাস বদলালেই ভেঙে পড়ে। **Reciprocal Rank Fusion** মান নয়, **র্যাংক** মেশায় – তাই একটি মাত্র ধ্রুবক লাগে এবং স্কেলের পার্থক্যে ভাঙে না।

```
src/nutrivision/rag/retrieve.py
```

```
RRF_K = 60

# দুই র্যাংকিং থেকে একটিই মিলিত স্কোর
fused[idx] = fused.get(idx, 0.0) + 1.0 / (RRF_K + rank)
```

কেন Cross-Encoder Rerank?

BM25 ও dense – দুটোই প্রশ্ন ও নথিকে **আলাদাভাবে** স্কোর করে। Cross-encoder দুটিকে **একসাথে** পড়ে। এটি সেই ভুল ধরে যেখানে "এতে কত সোডিয়াম" প্রশ্নে **ভুল খাবারের** সোডিয়াম-নথি চলে আসে – যা শব্দগত ও অর্থগতভাবে প্রায় অভিন্ন, কিন্তু সিদ্ধান্তগতভাবে সম্পূর্ণ ভুল।

Dish Conditioning – এই সিস্টেমের নিজস্ব বৈশিষ্ট্য

সাধারণ RAG শুধু প্রশ্নের লেখা থেকে শুরু করে। এখানে **ছবি-মডেল আগেই খাবারের নাম জানিয়ে দিয়েছে**। তাই "এতে কত সোডিয়াম?" প্রশ্নটিকে আগে "sushi-তে কত সোডিয়াম?" বানিয়ে নেওয়া হয় – নইলে ১০১টি প্রায়-অভিন্ন সোডিয়াম-নথির মধ্যে সঠিকটি বাছাই অসম্ভব।

১৪.৩ GraphRAG – জ্ঞানের জাল

খাবার → উপকরণ সম্পর্ক দিয়ে একটি গ্রাফ বানানো হয়েছে:

১৮১	নোড (৬০টি খাবার + ১২১টি উপকরণ)
৩২৩	এজ (ওজনসহ, গ্রাম-ভিত্তিক)
৫৮	একাধিক খাবারে ব্যবহৃত উপকরণ

এটি এমন প্রশ্নের উত্তর দেয় যা **কোনো একক নথিতে নেই** – যেমন "কোন কোন খাবারে আখরোট আছে?" এটি ৬০টি আলাদা উপকরণ-তালিকার **উল্টো অনুসন্ধান**।

১৪.৪ Grounding – সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা

নিয়ম
উত্তরে থাকা **প্রতিটি সংখ্যা** প্রকাশের আগে প্রাপ্ত নথির সাথে মিলিয়ে দেখা হয়। না মিললে সেই উত্তর বাতিল। ফলে মডেল সংখ্যা **বানাতে পারে না**।

এবং জ্ঞানভাণ্ডারের বাইরের প্রশ্ন (যেমন "আবহাওয়া কেমন?") ভাষা-মডেলে **পাঠানোই হয় না** – ১৩০ মিলিসেকেন্ডেরও কমে সরাসরি প্রত্যাখ্যান করা হয়। এতে খরচও বাঁচে।

১৫. ধাপ ১০: RAG মূল্যায়ন

৭৬টি প্রশ্নের একটি **gold set** হাতে তৈরি করা হয়েছে, যার সঠিক উত্তর আগে থেকেই জানা।

১৫.১ Retrieval মান

প্রশ্নের ধরন	n	recall@1	recall@5	MRR
nutrient_lookup	৪৮	৬৪.৬	১০০.০	০.৮২৩
ingredients	১০	১০০.০	১০০.০	১.০০০
graph_inversion	৮	৭৫.০	৮৭.৫	০.৮১২
superlative	৪	১০০.০	১০০.০	১.০০০

১৫.২ উত্তরের মান

ধরন	n	সঠিক	grounded	প্রত্যাখ্যান	p50 সময়
nutrient_lookup	৪৮	৯৭.৯%	১০০%	০%	১২৬২ ms
ingredients	১০	১০০%	১০০%	০%	১৬০৮ ms
graph_inversion	৮	১০০%	১০০%	০%	১৫৩৪ ms
superlative	৪	১০০%	১০০%	০%	১৮৬৬ ms
out_of_scope	৬	১০০%	১০০%	১০০%	৭০ ms

সারাংশ

সামগ্রিক সঠিকতা **৯৮.৫৭%** · grounded **১০০%** · ভুল প্রত্যাখ্যান **০টি** · সম্পূর্ণ মূল্যায়নের খরচ **\$০.০২৪** · সময় **৯৮.৮** সেকেন্ড।

লক্ষণীয়: সীমার বাইরের প্রশ্ন ৭০ ms-এ প্রত্যাখ্যাত – কারণ সেখানে OpenAI ডাকাই হয় না।

১৬. ব্যাকএন্ড: FastAPI

Python-এর FastAPI দিয়ে API সার্ভার, Docker-এ Hugging Face Spaces-এ চলে।

এন্ডপয়েন্ট	মেথড	কাজ
/predict	POST	ছবি → শ্রেণি + আত্মবিশ্বাস + conformal সেট + পুষ্টি
/explain	POST	Grad-CAM তাপচিত্র
/ask	POST	RAG প্রশ্নোত্তর
/history	GET / DELETE	ব্যবহারকারীর নিজের রেকর্ড
/feedback	POST	ভুল সংশোধনের প্রতিক্রিয়া
/stats , /analytics	GET	অ্যাডমিন পরিসংখ্যান
/health , /warm	GET / POST	সার্ভার প্রস্তুতি

১৬.১ নিরাপত্তা

টোকেন সার্ভারেই যাচাই

ব্যবহারকারী Google/ইমেইল দিয়ে লগইন করলে Firebase একটি **ID টোকেন** দেয়। সেটি ব্যাকএন্ডে Google-এর পাবলিক কী দিয়ে **যাচাই** করা হয়। ফলে ব্যবহারকারীর পরিচয় আসে **স্বাক্ষর** থেকে, অনুরোধের ভেতরে লেখা কোনো মান থেকে নয় – তাই কেউ অন্যের ইতিহাস পড়তে পারে না।

১৬.২ Next.js প্রক্সি স্তর

ব্রাউজার সরাসরি ব্যাকএন্ডে যায় না; Next.js-এর API route দিয়ে যায়।

```
web/lib/upstream.ts
```



```

export const UPSTREAM = process.env.FOODGENOME_API;

export function forwardAuth(request: Request, extra: HeadersInit = {}): Headers {
  const headers = new Headers(extra);
  const auth = request.headers.get("authorization");
  if (auth) headers.set("Authorization", auth); // টোকেন সামনে পাঠানো
  return headers;
}

export async function relay(res: Response, label: string) {
  // 401/403 নিজের রূপেই পৌঁছাতে হবে – 502-এ চেপে দিলে
  // "লগইন করুন" আর "সার্ভার নষ্ট" এক হয়ে যায়
  if (res.status === 401 || res.status === 403) {
    const body = await res.json().catch(() => ({}));
    return Response.json({ error: body.detail ?? "Sign in first." },
      { status: res.status });
  }
  if (!res.ok) return Response.json({ error: `${label} returned ${res.status}.` },
    { status: 502 });
  return Response.json(await res.json());
}

```

`FOODGENOME_API`-তে `NEXT_PUBLIC_` উপসর্গ **নই** – তাই এটি শুধু সার্ভারে পড়া হয়, ব্রাউজার ব্যাকএন্ডের ঠিকানা কখনো দেখে না।

১৭. ফ্রন্টএন্ড: Next.js

Next.js 16 (App Router) + TypeScript + Tailwind CSS 4, Vercel-এ হোস্ট।

১৭.১ ডিজাইন ভাষা – কমিক প্রিন্ট

প্রচলিত gradient-নির্ভর SaaS চেহারার বদলে **ছাপা কমিক বইয়ের** ভাষা ব্যবহার করা হয়েছে:

- **Ben-Day dots** – পুরোনো ছাপার হাফটোন বিন্দু, CSS `radial-gradient` দিয়ে
- **Plate misregistration** – শিরোনামে লাল/নীল সামান্য সরানো, ছাপার ভুলের অনুকরণ
- **হার্ড ছায়া** – blur ছাড়া কঠিন অফসেট ছায়া
- **প্যানেল** – মোটা কালো বর্ডারে কমিক প্যানেল

১৭.২ থিম-সচেতন ক্যানভাস মাকড়সার জাল

হিরো অংশে একটি সত্যিকারের মাকড়সার জাল আঁকা হয়, যা মাউসের নড়াচড়ায় কাঁপে।

`web/app/components/SpiderWebBackground.tsx`

```
// Canvas CSS variable পড়তে পারে না, তাই document থেকে resolve করতে হয়
const readToken = (value: string, fallback: string) => {
  const token = value.match(/^var\((\s*(--[w-]+\s*)\)$/);
  if (!token) return value;
  const resolved = getComputedStyle(document.documentElement)
    .getPropertyValue(token[1]).trim();
  return resolved || fallback;
};
```

কেন এটি দরকার ছিল

সাইটের `--line` রং light থিমে প্রায় কালো (`#0b0b0f`) আর dark থিমে ক্রিম (`#f4f1e8`) – সম্পূর্ণ উল্টে যায়।
কোনো নির্দিষ্ট hex লিখলে যেকোনো একটি থিমে জালটি **অদৃশ্য** হয়ে যেত।

স্প্রিং-ভিত্তিক কাঁপুনি

```
// প্রতিটি বিন্দু তার বিশ্রাম-অবস্থানের চারপাশে একটি স্প্রিং
const tx = v.bx + px + sway;
const ty = v.by + py + sway * 0.6;
v.vx = (v.vx + (tx - v.x) * 0.09) * 0.86; // 0.86 = damping
v.vy = (v.vy + (ty - v.y) * 0.09) * 0.86;
v.x += v.vx;
v.y += v.vy;
```

একটি বাস্তব বাগ যা ধরা পড়েছিল

`origin = [1, 0]` — এই ডিফল্ট প্রতিটি render-এ **নতুন array** তৈরি করে। `useEffect`-এর dependency-তে `origin` রাখায় প্রতিবার নতুন identity হওয়ায় effect বারবার ভেঙে-গড়ে যাচ্ছিল — নতুন lattice, নতুন listener, প্রতিটি render-এ। সমাধান: array ভেঙে `originX`, `originY` সংখ্যা দুটিকে dependency করা।

১৭.৩ ত্রিমাত্রিক (3D) চরিত্র

Three.js + React Three Fiber দিয়ে একটি rigged 3D চরিত্র প্রতিটি পাতায় দেখানো হয়।

Toon shading ও কালি-রেখা

```
// চারটি সমতল ধাপ — এটিই আলোকে cel-shading বানায়
const steps = new Uint8Array([88, 148, 204, 255]);
const ramp = new THREE.DataTexture(steps, steps.length, 1, THREE.RedFormat);
ramp.minFilter = THREE.NearestFilter;
ramp.magFilter = THREE.NearestFilter;
```

যে বাগটি দুবার ঠিক করতে হয়েছে

কালি-রেখার জন্য মডেলের একটি অনুলিপি উল্টো দিক থেকে (BackSide) সামান্য বড় করে আঁকা হয়। প্রথমে `scale.multiplyScalar(1.035)` ব্যবহার করা হয়েছিল — এটি কেবল তখনই কাজ করে যখন geometry তার নিজের origin-এ কেন্দ্রীভূত। যে মডেলের অঙ্গগুলো origin থেকে দূরে, সেখানে এটি প্রতিটি অঙ্গকে **বাইরে ছুঁড়ে দেয়** — হাতের পেছনে কালো দাগ তৈরি হয়েছিল। সমাধান: view-space-এ প্রতিটি vertex-কে তার **normal বরাবর** সরানো।

```
vertexShader: `
  uniform float uThickness;
  void main() {
    vec4 view = modelViewMatrix * vec4( position, 1.0 );
    vec3 n = normalize( normalMatrix * normal );
    view.xyz += n * uThickness * -view.z;    // গভীরতা অনুযায়ী সমান পুরুত্ব
    gl_Position = projectionMatrix * view;
  }`
```

যেকোনো মডেল গ্রহণযোগ্য করা

বিভিন্ন শিল্পী বিভিন্ন স্কেল, origin ও up-axis-এ মডেল রপ্তানি করেন। তাই drei-র `Resize` ও `Center` ব্যবহার করা হয়েছে, যাতে নতুন মডেল বসালে ক্যামেরা নতুন করে ঠিক করতে না হয়।

১৭.৪ শব্দ — সংশ্লেষিত, ফাইল নয়

"থুইপ" শব্দটি কোনো অডিও ফাইল থেকে আসে না — Web Audio API দিয়ে **গণনা করে** বানানো হয়।

```
web/lib/sfx.ts
```

```
// স্তর ১: হিস্ শব্দ – noise-এর উপর দ্রুত নিচে নামা bandpass
band.frequency.setValueAtTime(4200 * vary, now);
band.frequency.exponentialRampToValueAtTime(620, now + 0.16);

// স্তর ২: নিচে নামা সুর – জাল ছুটে যাওয়ার অনুভূতি
tone.type = "triangle";
tone.frequency.setValueAtTime(880 * vary, now);
tone.frequency.exponentialRampToValueAtTime(170, now + 0.13);
```

ব্রাউজার ব্যবহারকারীর স্পর্শ ছাড়া অডিও চালু করতে দেয় না, তাই প্রথম ক্লিকে `AudioContext` খোলা হয়। ডিফল্ট শব্দ **বন্ধ** – হেডারের বোতাম দিয়ে চালু করতে হয়।

১৮. ডিপ্লয়মেন্ট

অংশ	প্ল্যাটফর্ম	বিবরণ
ফ্রন্টএন্ড	Vercel	Next.js, git push-এ স্বয়ংক্রিয় ডিপ্লয়
ব্যাকএন্ড	Hugging Face Spaces	Docker, CPU, ONNX ইনফারেন্স
ডেটাবেস	Firebase Firestore	ব্যবহারকারীর ইতিহাস

```
# পরিবেশ চলক
FOODGENOME_API = https://asifzaman1912-prac.hf.space # Vercel-এ
OPENAI_API_KEY = ... # শুধু ব্যাকএন্ড
REQUIRE_AUTH = 1
ADMIN_EMAILS = ...
```

মডেল অপটিমাইজেশন

৩D মডেলটি প্রথমে ২০.৩ MB ছিল। টেক্সচার ১০২৪ পিক্সেলে ছোট করে, WebP-তে রূপান্তর করে এবং Draco দিয়ে geometry সংকুচিত করে **২.১ MB**-তে নামানো হয়েছে – ৯০% হ্রাস। Draco ডিকোডারও নিজের সার্ভার থেকে দেওয়া হয়, Google CDN থেকে নয়।

১৯. সম্পূর্ণ ফলাফল

১৯.১ শ্রেণিকরণ

মডেল	test top-1	test top-5
DINOv2-L probe	৯৪.৮৭	৯৯.২০
EVA-02-L probe	৯৫.৫৩	৯৯.৩০
EVA-02 fine-tuned	৯৫.৯০	৯৯.৫২
SigLIP-SO400M probe	৯৬.৮৩	৯৯.৫৩
Gated Fusion	৯৬.৯৭	৯৯.৪৮
Ensemble (গড়) — চূড়ান্ত	৯৭.১৬	৯৯.৬৪

১৯.২ নির্ভরযোগ্যতা

ECE (calibration-এর আগে)	০.০৫১৩৫
ECE (পরে)	০.০০৬২৪
Conformal coverage (৯৯% লক্ষ্য)	৯৯.৬১%
গড় সেট আকার	১.৩৯

১৯.৩ RAG

সামগ্রিক সঠিকতা	৯৮.৫৭%
Grounded উত্তর	১০০%
প্রত্যাখ্যানের নির্ভুলতা	১০০%
ভুল প্রত্যাখ্যান	০
সম্পূর্ণ মূল্যায়নের খরচ	\$০.০২৪

২০. সীমাবদ্ধতা

একটি সং প্রজেক্টে সীমাবদ্ধতা বলা আবশ্যিক:

- মাত্র ১০১টি শ্রেণি।** এর বাইরের খাবার দেখালে সিস্টেম ভুল উত্তর দেবে বা প্রত্যাখ্যান করবে।
- পরিমাণ অনুমান করা হয় না।** ছবি থেকে খাবারের ওজন মাপা হয় না; একটি আদর্শ পরিবেশন ধরে নেওয়া হয়। প্লেটে দ্বিগুণ খাবার থাকলে সংখ্যা দ্বিগুণ হওয়া উচিত ছিল।

3. **একটি প্লেটে একটি খাবার।** একাধিক পদ থাকলে শুধু প্রধানটিই শনাক্ত হয়।
4. **Composite রেসিপি আনুমানিক।** ৬০টি খাবারের পুষ্টি সাধারণ রেসিপি থেকে গণনা করা – নির্দিষ্ট রেস্টোরাঁর রান্নার সাথে ভিন্ন হতে পারে।
5. **Conformal গ্যারান্টি শর্তসাপেক্ষ।** calibration ও test ডেটা exchangeable হলেই কেবল প্রযোজ্য। খুব ভিন্ন ধরনের ছবিতে গ্যারান্টি ভেঙে যেতে পারে।
6. **চিকিৎসা পরামর্শ নয়।** এটি শিক্ষামূলক প্রকল্প।

২১. পরিভাষা

শব্দ	বাংলা ব্যাখ্যা
Backbone	বড় pretrained মডেল যা ছবি থেকে বৈশিষ্ট্য বের করে
Embedding / Feature	ছবির অর্থ ধারণকারী সংখ্যার ভেক্টর
Probe head	জমানো ভেক্টরের উপর প্রশিক্ষিত ছোট নেটওয়ার্ক
Fine-tuning	পুরো pretrained মডেলের ওজন নতুন কাজের জন্য শেখানো
Ensemble	একাধিক মডেলের ভবিষ্যদ্বাণী একত্র করা
Calibration	আত্মবিশ্বাসের সংখ্যাকে বাস্তবের সাথে মেলানো
ECE	ঘোষিত আত্মবিশ্বাস ও প্রকৃত নির্ভুলতার গড় পার্থক্য
Conformal prediction	গাণিতিক নিশ্চয়তাসহ প্রার্থী-সেট তৈরি
Coverage	সত্য উত্তর সেটের ভেতরে থাকার হার
Grad-CAM	মডেল ছবির কোথায় তাকিয়েছে তার তাপচিত্র
RAG	তথ্য খুঁজে এনে সেই তথ্যের ভিত্তিতে উত্তর তৈরি
BM25	শব্দ-মিল ভিত্তিক ক্লাসিক অনুসন্ধান অ্যালগরিদম
RRF	দুই র‍্যাংকিং তালিকা মেশানোর পদ্ধতি
Cross-encoder	প্রশ্ন ও নথি একসাথে পড়ে প্রাসঙ্গিকতা যাচাই করে
Grounding	উত্তরের প্রতিটি দাবি উৎসের সাথে মিলিয়ে দেখা
McNemar test	দুই মডেলের পার্থক্য অর্থবহ কি না তা পরীক্ষা
Mixup	দুই নমুনা মিশিয়ে নতুন প্রশিক্ষণ-উদাহরণ
EMA	ওজনের চলমান গড়, যা প্রশিক্ষণ স্থিতিশীল করে
ONNX	মডেল বিনিময়ের মানক ফরম্যাট, দ্রুত CPU ইনফারেন্স
Draco	3D geometry সংকোচনের পদ্ধতি

FoodGenome AI – সম্পূর্ণ প্রযুক্তিগত ব্যাখ্যা
সব সংখ্যা `artifacts/reports/`-এর পরিমাপ থেকে নেওয়া, হাতে লেখা নয়।